POLITECHNIKA BYDGOSKA

im. Jana i Jędrzeja Śniadeckich

WYDZIAŁ TELEKOMUNIKACJI, INFORMATYKI

I ELEKTROTECHNIKI



na kierunku Informatyka stosowana

CYBERBECPIECZEŃSTWO

PROJEKT

Atak słownikowy

Pracę wykonał:

Naskręt Cezary

Data:

2023-11-19

Spis treści

1. Cel projektu 3
2. Jak działa atak słownikowy 3
3. User Stories 4
4. Aplikacja 5
5. Przygotowanie słownika 6
6. Próby logowania 7
7. Wielokrotne próby logowania 10
8. Wprowadzenie zabezpieczeń 12

1. Cel projektu

Celem projektu jest stworzenie prostego systemu, który będzie służył jako testowa aplikacja. Ta aplikacja powinna umożliwiać użytkownikom rejestrację, logowanie. Aplikacja ta będzie poddana testom pod kątem ataków słownikowych.

Po przygotowaniu aplikacji, projekt skupi się na przeprowadzeniu testów, które symulują próby ataków słownikowych na hasła użytkowników. Celem jest zidentyfikowanie ewentualnych słabości w mechanizmach zabezpieczeń aplikacji oraz ocena jej zdolności do obrony przed takimi atakami. Jest to główny cel projektu.

Po przeprowadzeniu testów ocenię wyniki i zidentyfikuje słabości. Następnie przystąpię do wprowadzenia odpowiednich ulepszeń w aplikacji, które zwiększą jej odporność na ataki słownikowe.

Podczas trwania projektu zostanie przygotowana dokumentacja obejmująca zarówno kroki przeprowadzonych testów, jak i udoskonalenia wprowadzone w aplikacji w celu zwiększenia jej bezpieczeństwa. Raport ten będzie stanowić podsumowanie projektu.

2. Jak działa atak słownikowy

Cały proces możemy podzielić na cztery etapy.

Przygotowanie słownika: Atakujący przygotowuje listę potencjalnych haseł, często opartą na słowach, które mogą być związane z celem ataku. Słownik ten może zawierać popularne hasła, słowa ze słownika, nazwy firm, imiona, słowa kluczowe itp.

Próby logowania: Atakujący używa słownika do automatycznego przetestowania każdego hasła na liście w celu zalogowania się do systemu, konta lub serwisu. Dla każdej próby hasło z listy jest podawane w pole hasła, a potem próbuje się dostęp do systemu.

Próby wielokrotne: Atak słownikowy zazwyczaj jest prowadzony jako seria prób. Atakujący kontynuuje próby, przekształcając kolejne hasła z listy, dopóki nie uzyska dostępu lub wyczerpie całą listę.

Monitorowanie odpowiedzi: W miarę jak atakujący próbuje różne hasła, monitoruje odpowiedzi systemu. Jeśli odgadnie poprawne hasło, może uzyskać dostęp do systemu lub konta.

3. User Stories

Jako tester bezpieczeństwa, chciałbym mieć możliwość przetestowania odporności aplikacji na słownikowe ataki, aby zidentyfikować słabe hasła i udoskonalić nasze procedury zabezpieczeń.

Jako użytkownik aplikacji, oczekuję, że moje hasło zostanie przetestowane pod kątem słabej konfiguracji, aby upewnić się, że moje konto jest chronione przed atakami hakerskimi.

Jako tester bezpieczeństwa, mogę określić kryteria wyboru słownika używanego do ataku, aby dostosować testy do konkretnych wymagań projektu.

Jako użytkownik, mogę potrzebuje zabezpieczenia swojego konta przed próbami ataku na moje hasło, takie jak blokowanie dostępu po kilku nieudanych próbach logowania.

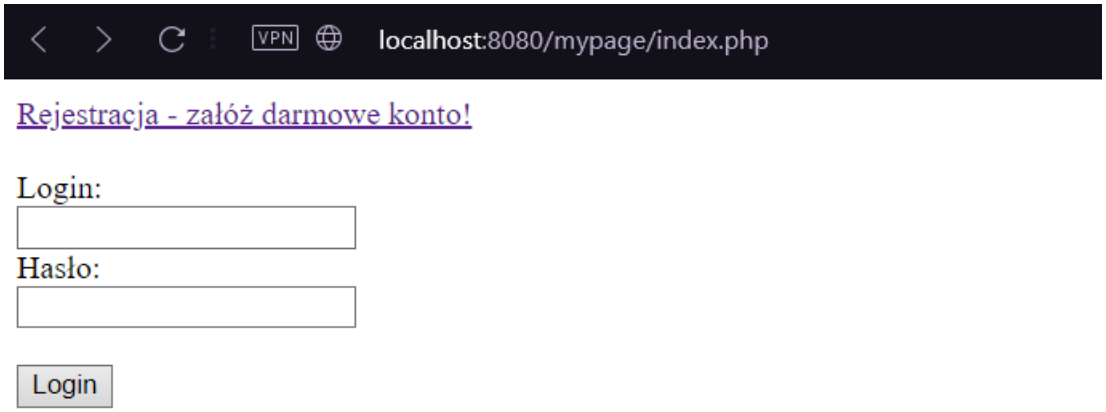
Jako użytkownik, mogę wybrać bardziej złożone i trudniejsze do odgadnięcia hasło, a system powinien ocenić jego siłę i zgodność z politykami bezpieczeństwa.

Jako członek zespołu projektowego, chciałbym mieć dostęp do narzędzi do testowania odporności aplikacji na ataki słownikowe, aby przetestować nowe funkcje pod kątem bezpieczeństwa.

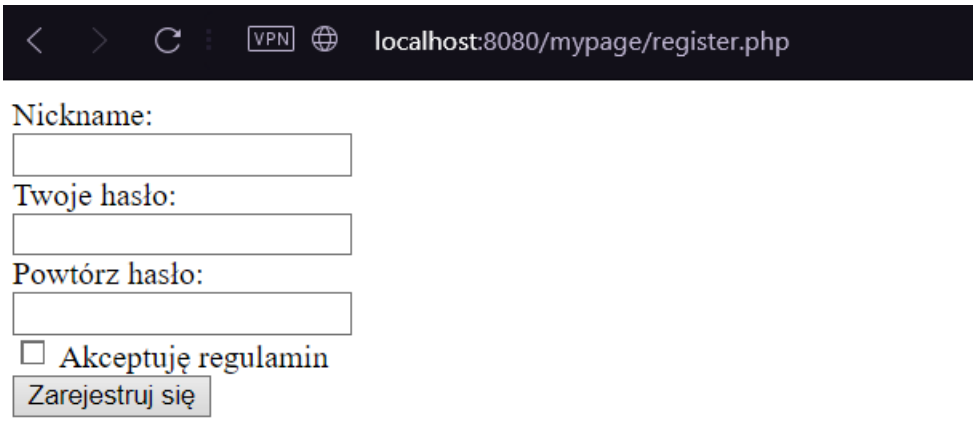
Jako użytkownik, oczekuje, że system dostarczy mi wskazówki i zalecenia dotyczące tworzenia silnych haseł, aby zwiększyć poziom bezpieczeństwa mojego konta.

4. Aplikacja

Zaimplementowałem prostą aplikację która umożliwia logowanie.

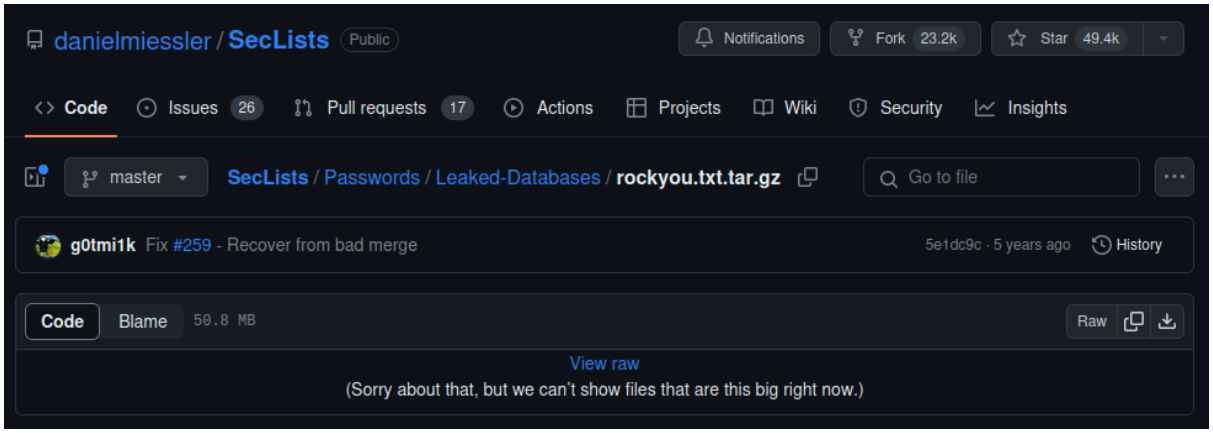


Stworzyłem też formularz rejestrowania nowego konta.

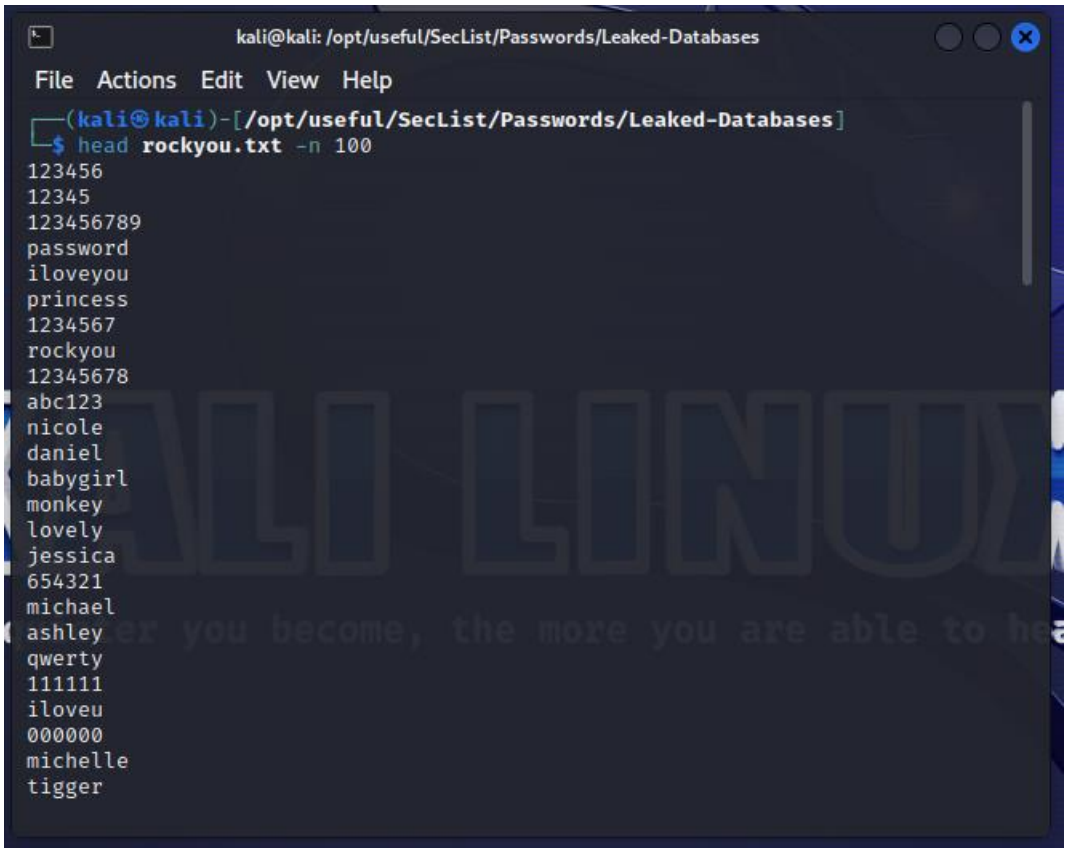


5. Przygotowanie słownika

Słownik który wykorzystałem nazywa się SecList. Posiada ona wiele różnych list loginów i haseł np. do serwisów społecznościowych lub np. routerów itp.

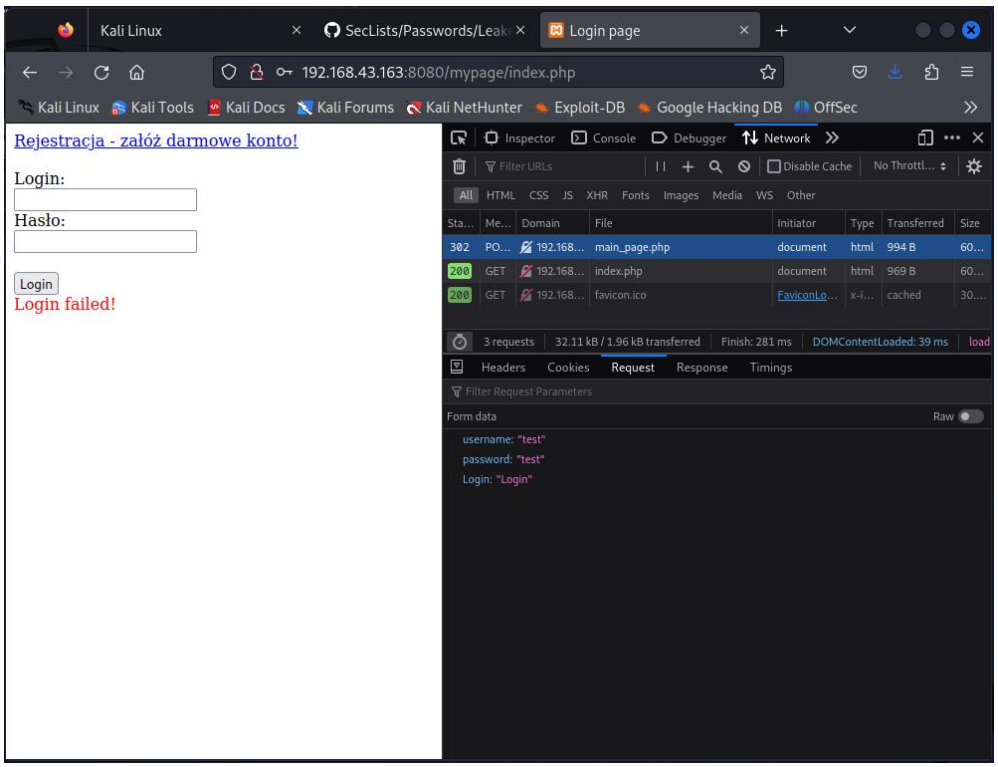


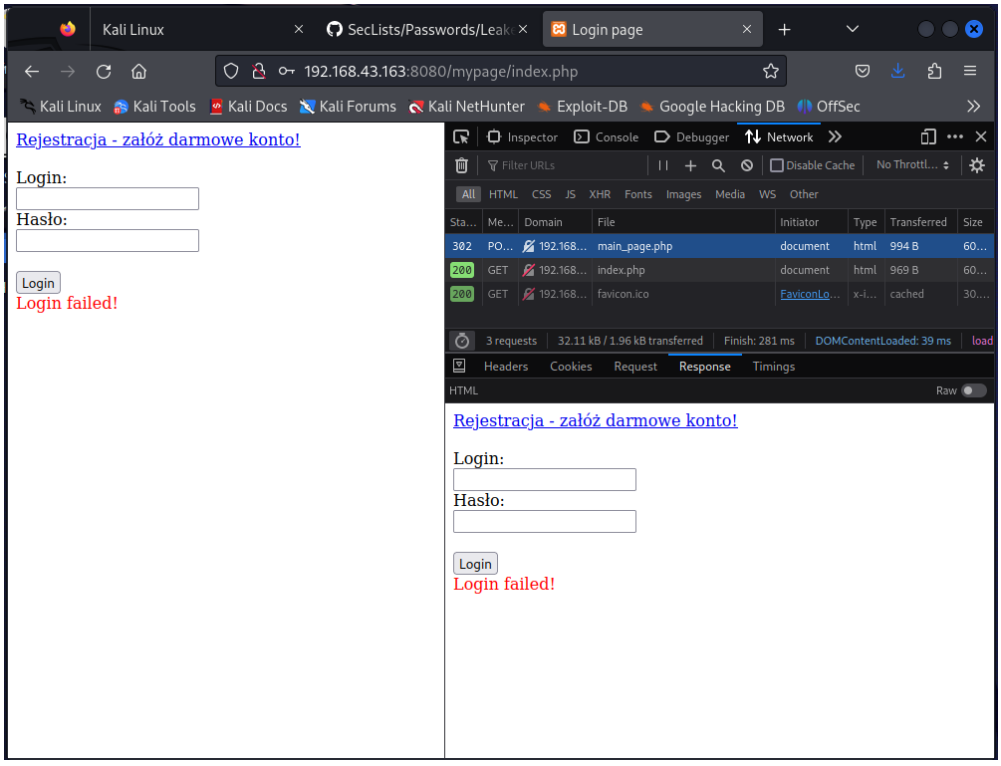
Poniżej zaprezentowałem 100 najpopularniejszych haseł z listy która została stworzona na podstawie wycieków baz danych.

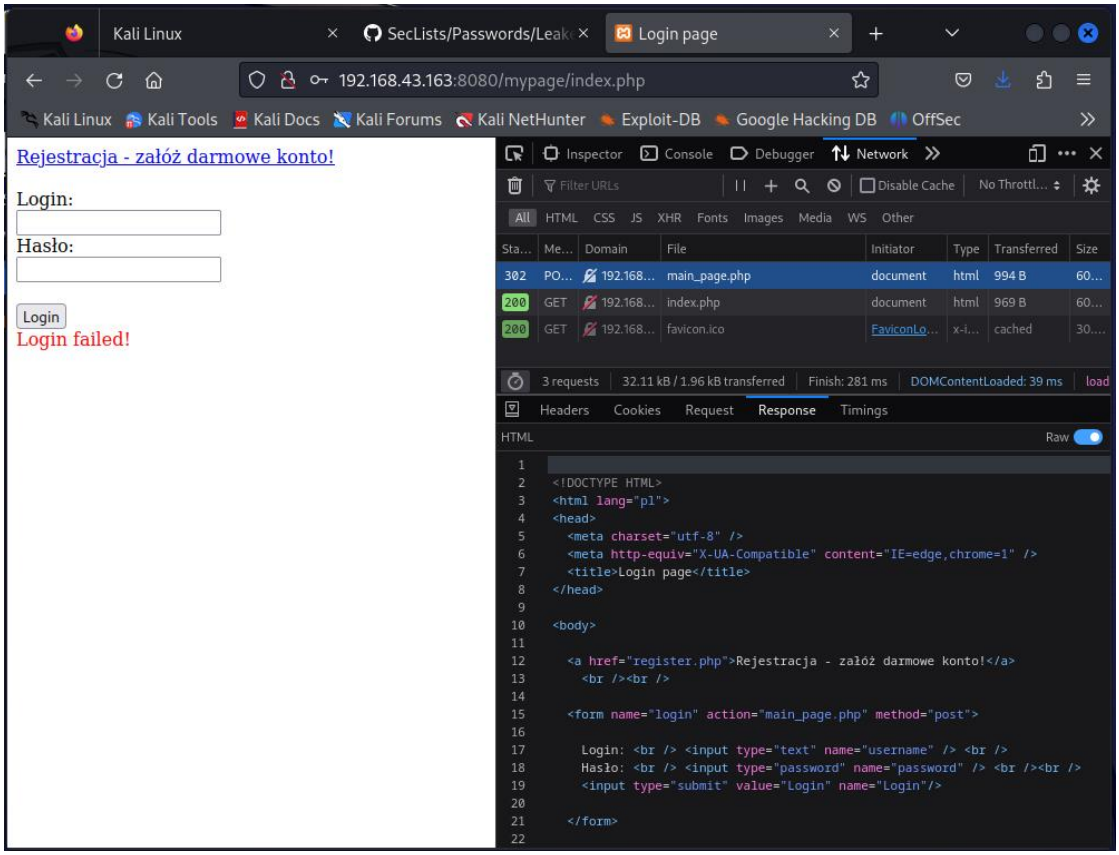


6. Próby logowania

Na początku wykonujemy próby logowania bez żadnych narzędzi stosujących słownik. Dzięki temu możemy zdobyć wiele informacji takich jak format zapytania jakie jest wysyłane z naszej przeglądarki do serwisu. Dowiadujemy się też jak wygląda odpowiedź w przypadku błędnego i poprawnego logowania.

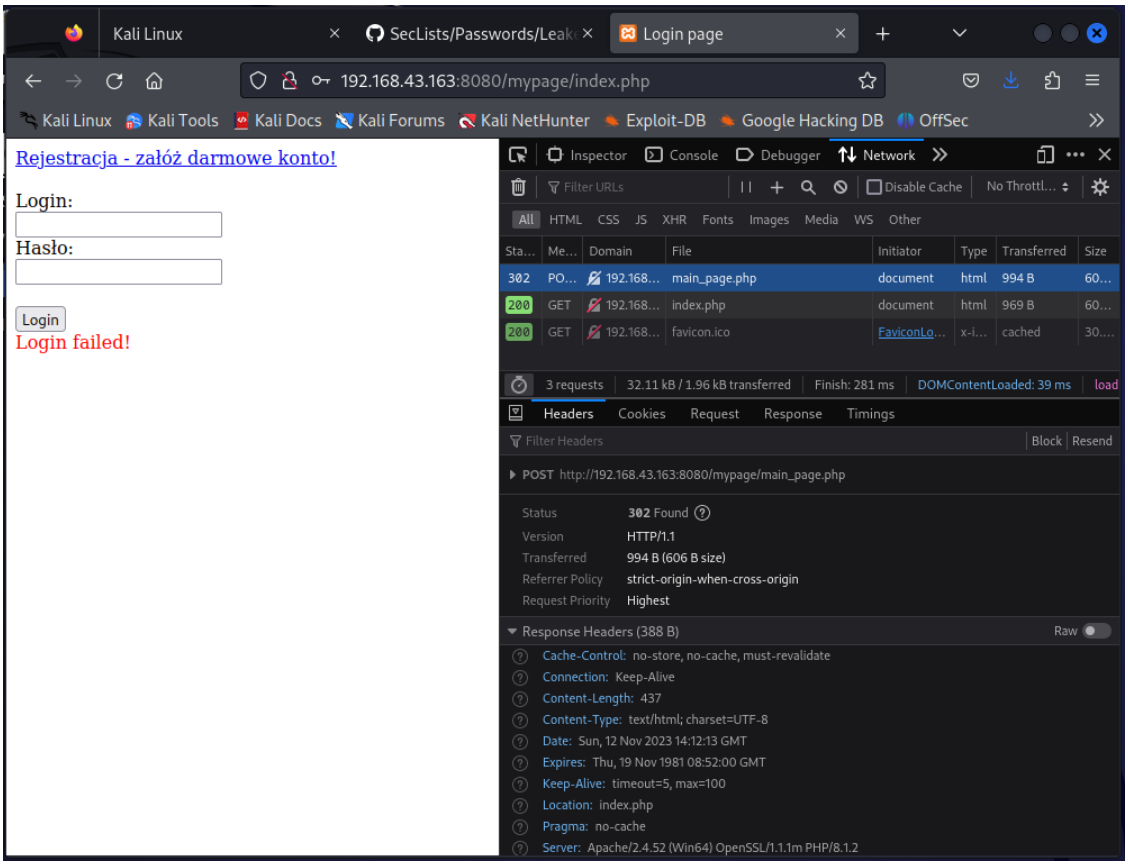


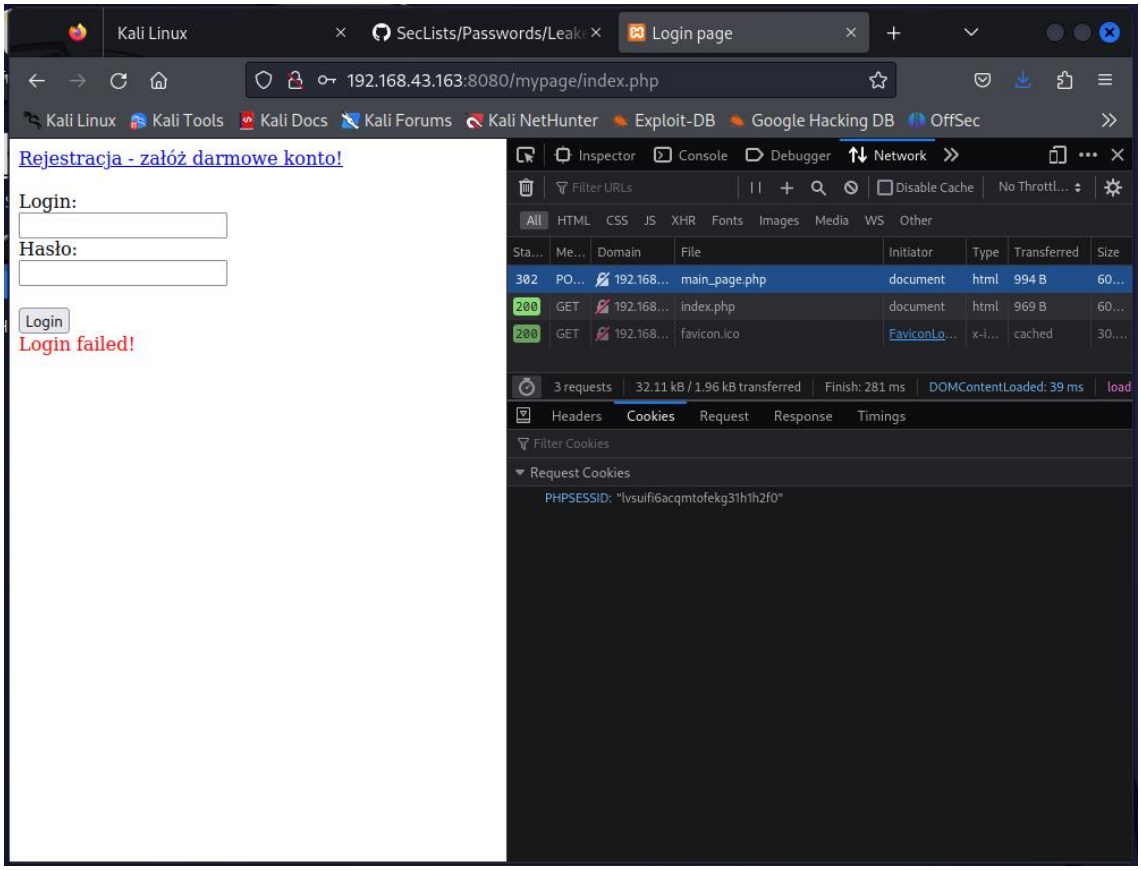


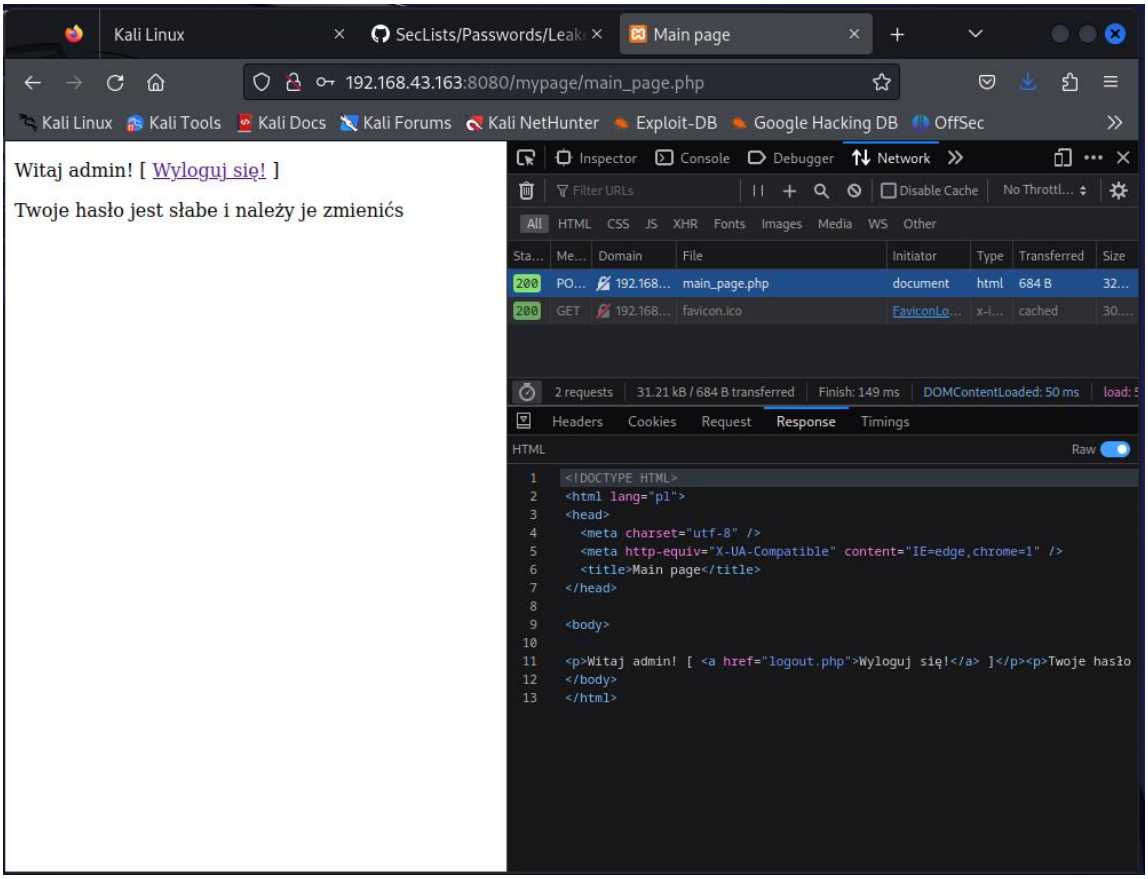

Kali Linux
SecLists/Passwords/Leak
Login page
192.168.43.163:8080/mypage/index.php
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec
Rejestracja - załóż darmowe konto!
Login:
Hasło:
Login
Login failed!
Inspector Console Debugger Network
Filter URLs Disable Cache No Throttl...
All HTML CSS JS XHR Fonts Images Media WS Other
Sta... Me... Domain File Initiator Type Transferred Size
302 PO... 192.168... main_page.php document html 994 B 60...
200 GET 192.168... index.php document html 969 B 60...
200 GET 192.168... favicon.ico FaviconLo... x-i... cached 30....
3 requests 32.11 kB / 1.96 kB transferred Finish: 281 ms DOMContentLoaded: 39 ms load
Headers Cookies Request Response Timings
HTML Raw
<!DOCTYPE HTML>
<html lang="pl">
<head>
<meta charset="utf-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1" />
<title>Login page</title>
</head>
<body>
Rejestracja - załóż darmowe konto!

<form name="login" action="main_page.php" method="post">
Login:
 <input type="text" name="username" />
Hasło:
 <input type="password" name="password" />

<input type="submit" value="Login" name="Login"/>
</form>


Kali Linux
SecLists/Passwords/Leak
Login page
192.168.43.163:8080/mypage/index.php
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec
Rejestracja - załóż darmowe konto!
Login:
Hasło:
Login
Login failed!
Inspector Console Debugger Network
Filter URLs Disable Cache No Throttl...
All HTML CSS JS XHR Fonts Images Media WS Other
Sta... Me... Domain File Initiator Type Transferred Size
302 PO... 192.168... main_page.php document html 994 B 60...
200 GET 192.168... index.php document html 969 B 60...
200 GET 192.168... favicon.ico FaviconLo... x-i... cached 30....
3 requests 32.11 kB / 1.96 kB transferred Finish: 281 ms DOMContentLoaded: 39 ms load
Headers Cookies Request Response Timings
Filter Headers Block Resend
POST http://192.168.43.163:8080/mypage/main_page.php
Status 302 Found
Version HTTP/1.1
Transferred 994 B (606 B size)
Referrer Policy strict-origin-when-cross-origin
Request Priority Highest
Response Headers (388 B) Raw
Cache-Control: no-store, no-cache, must-revalidate
Connection: Keep-Alive
Content-Length: 437
Content-Type: text/html; charset=UTF-8
Date: Sun, 12 Nov 2023 14:12:13 GMT
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Keep-Alive: timeout=5, max=100
Location: index.php
Pragma: no-cache
Server: Apache/2.4.52 (Win64) OpenSSL/1.1.1m PHP/8.1.2


Kali Linux
SecLists/Passwords/Leak
Login page
192.168.43.163:8080/mypage/index.php
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec
Rejestracja - załóż darmowe konto!
Login:
Hasło:
Login
Login failed!
Inspector Console Debugger Network
Filter URLs
Disable Cache
No Throttl...
All HTML CSS JS XHR Fonts Images Media WS Other
Sta... Me... Domain File Initiator Type Transferred Size
302 PO... 192.168... main_page.php document html 994 B 60...
200 GET 192.168... index.php document html 969 B 60...
200 GET 192.168... favicon.ico FaviconLo... x-i... cached 30....
3 requests 32.11 kB / 1.96 kB transferred Finish: 281 ms DOMContentLoaded: 39 ms load
Headers Cookies Request Response Timings
Filter Cookies
Request Cookies
PHPSESSID: "lvsuifi6acqmtofekg31h1h2f0"

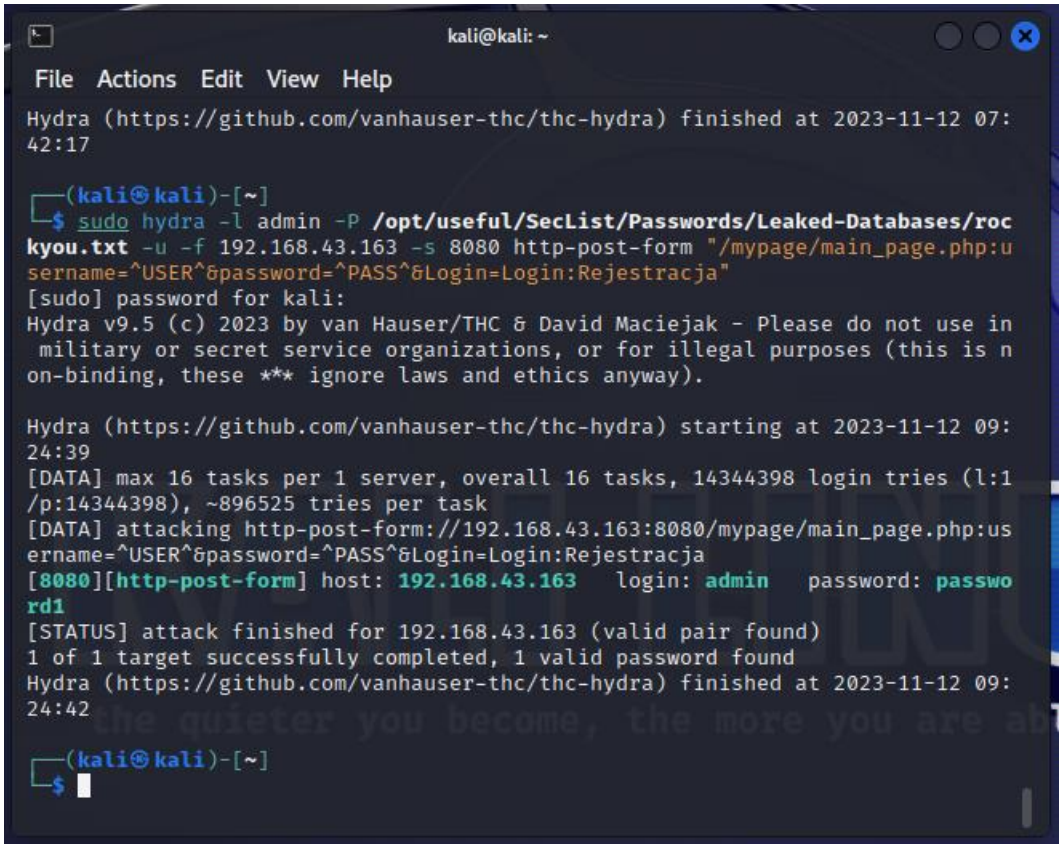

Kali Linux
SecLists/Passwords/Leak
Main page
192.168.43.163:8080/mypage/main_page.php
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec
Witaj admin! [Wyloguj się!]
Twoje hasło jest słabe i należy je zmienićs
Inspector Console Debugger Network
Filter URLs
Disable Cache
No Throttl...
All HTML CSS JS XHR Fonts Images Media WS Other
Sta... Me... Domain File Initiator Type Transferred Size
200 PO... 192.168... main_page.php document html 684 B 32...
200 GET 192.168... favicon.ico FaviconLo... x-i... cached 30....
2 requests 31.21 kB / 684 B transferred Finish: 149 ms DOMContentLoaded: 50 ms load:
Headers Cookies Request Response Timings
HTML
Raw
<!DOCTYPE HTML>
<html lang="pl">
<head>
<meta charset="utf-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1" />
<title>Main page</title>
</head>
<body>
Witaj admin! [Wyloguj się!] Twoje hasło
</body>
</html>

7. Wielokrotne próby logowania

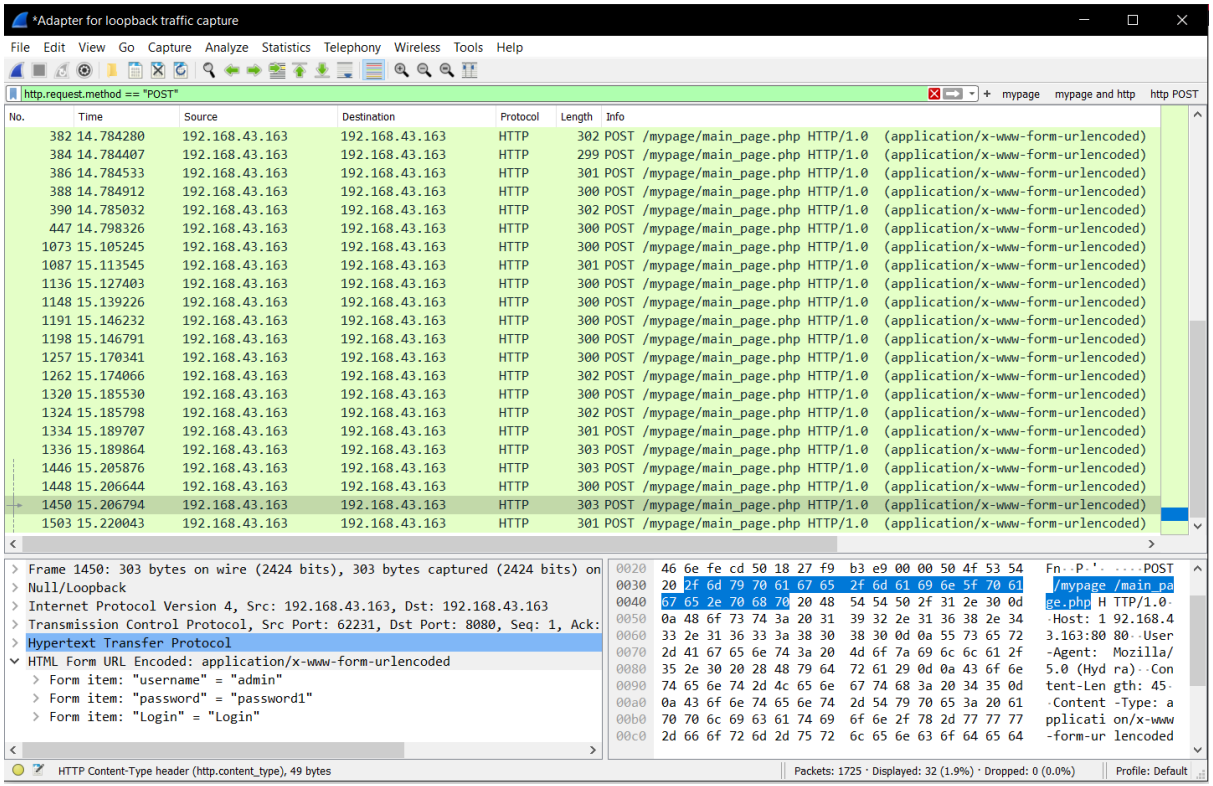
Do wielokrotnych prób logowanie nie wystarczy, już sama przeglądają. Do tego zadania można wykorzystać narzędzia takie jak Hydra.

Do działania hydra potrzebuje znać takie elementy jak adres atakowanej aplikacji. Nie musi być on w postaci adresu IP, może to być również adres domenowy. Potrzebujemy podać również port. Kolejnym wymaganym parametrem jest lista loginów, jakie mamy przetestować. W przykładzie poniżej skorzystałem z parametru -l admin do przetestowania haseł tylko dla tego loginu. Jeżeli mamy listę loginów do przetestowania to po parametrze -L musimy podać ścieżkę do pliku z listą loginów. Kolejny parametr to -P, po którym podaliśmy ścieżkę do listy haseł, jakie chcemy przetestować. Na samym końcu podajemy listę parametrów, jakie wcześniej widzieliśmy w formularzu. W miejsce loginu podajemy ^USER^, a w miejsce haseł podajemy ^PASS^. Na końcu podajemy ciąg znaków, jaki ma być wykryty po nieprawidłowej próbie logowania. Zamiast tego ciągu, możemy testować udaną próbę logowania. W takim przypadku musimy po parametrze ”S=” podać ciąg znaków, jaki ma być wykryty po udanej próbie logowania.

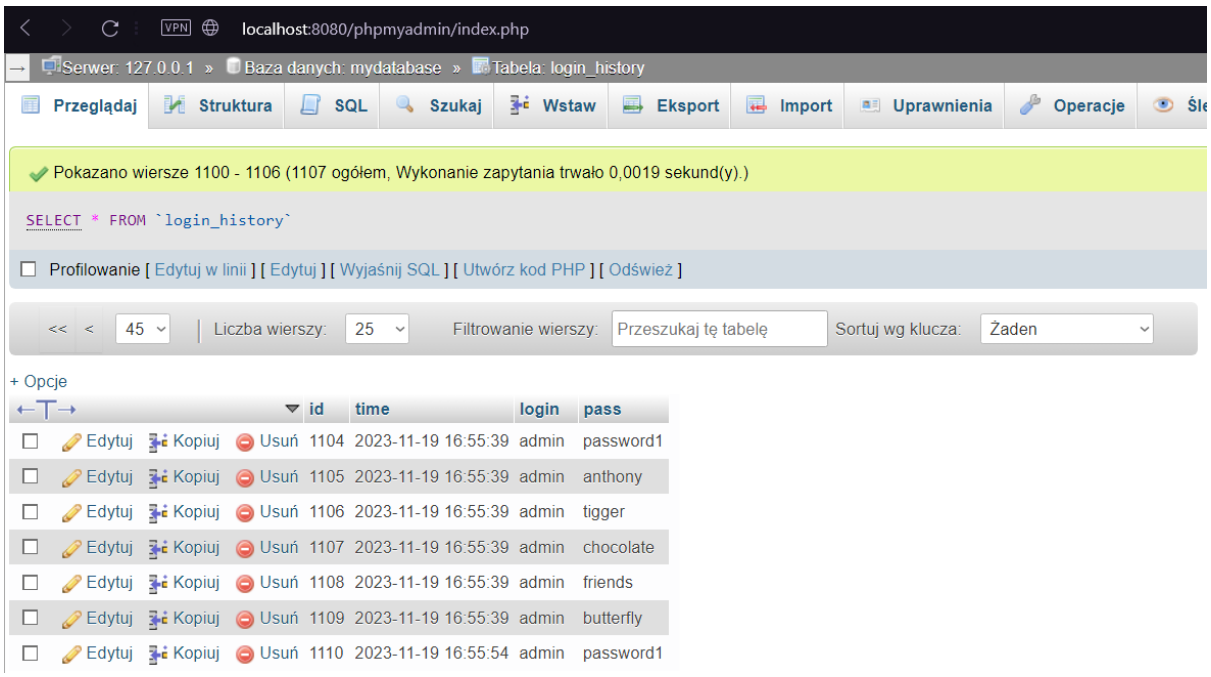
Poniżej widzimy, że dla loginu admin udało się znaleźć jedno hasło „password1”.



Podczas tego testu uruchomiłem również aplikację WireShark, która rejestrowała wszystkie pakiety wysyłane przez hydrę. Poniżej widzimy pakiet który wysyłał zapytanie, właśnie z wcześniej wykrytym już hasłem „password1”.

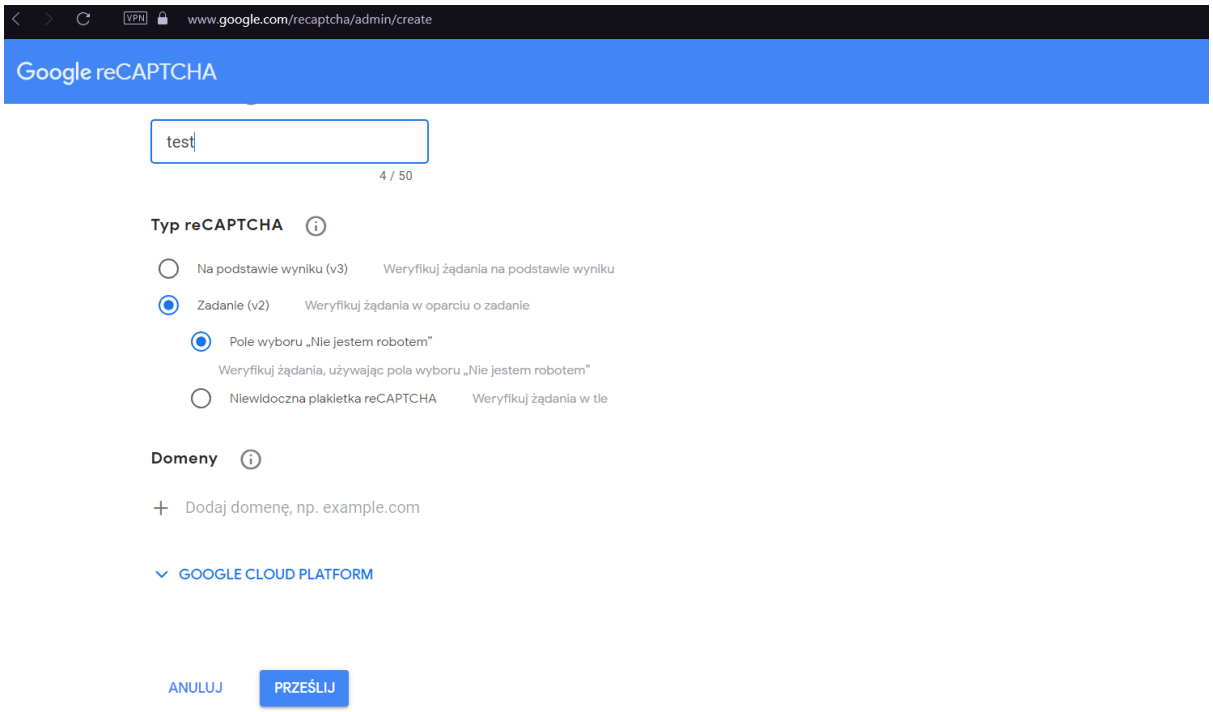


Stworzyłem również osobną tabelę w bazie danych, która rejestruje wszystkie próby logowania, dzięki czemu w późniejszej analizie możliwe jest wykrycie takiego ataku.

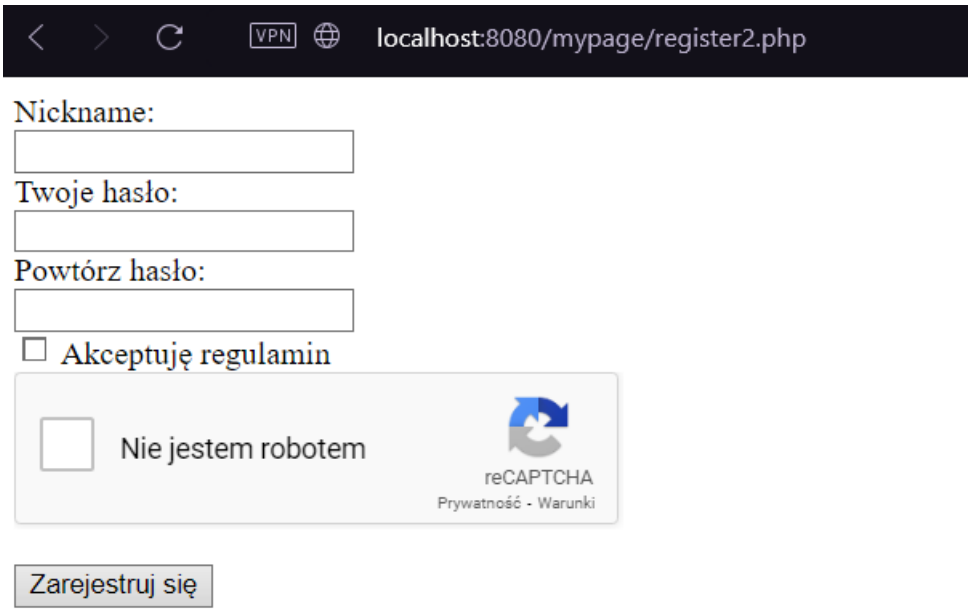


8. Wprowadzenie zabezpieczeń.

Najprostszym zabezpieczeniem, jakie możemy wprowadzić, jest reCAPTCHA. Każda osoba, która ma konto Google może wygenerować takie zabezpieczenie za darmo.



Do formularza dodawany jest jeden div. Podczas parsowania takiego formularza porównujemy otrzymany kod z tym, co otrzymaliśmy na stronie do generowania reCAPCHA'y.



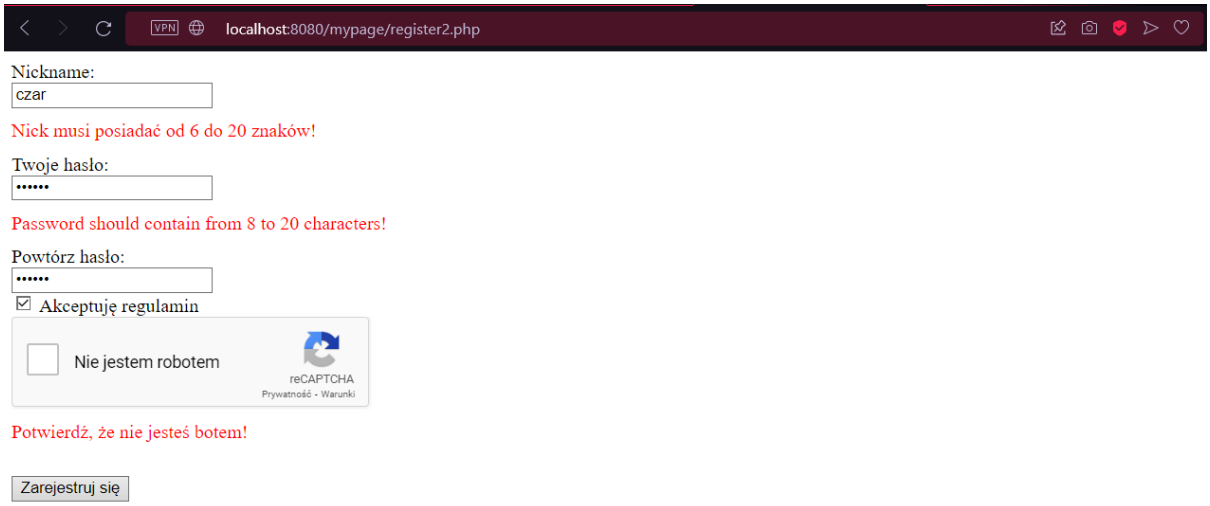
Kolejnym zabezpieczeniem, jakie wprowadziłem, jest samo przetestowanie jakości hasła wymyślonego przez użytkownika i podaniu informacji zwrotnej, jeżeli podane hasło okazało się za słabe.

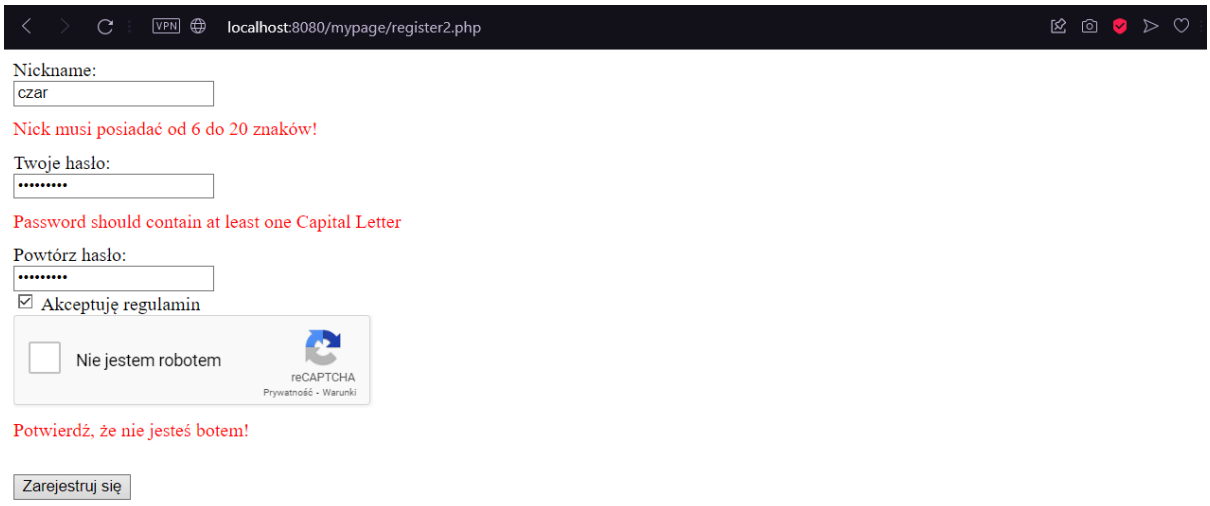
Długość hasła: Zalecane jest, aby hasło miało co najmniej 8 znaków. Dłuższe hasła są zazwyczaj bardziej bezpieczne.

Małe i wielkie litery: Hasło powinno zawierać zarówno małe, jak i wielkie litery.

Znaki specjalne: Zaleca się uwzględnienie przynajmniej jednego lub dwóch znaków specjalnych (np. !, @, #, $, %).

Cyfry: Hasło powinno zawierać przynajmniej jedną cyfrę.







Nickname:
czarek

Istnieje już konto o takim loginie!

Twoje hasło:
••••••••

Password should contain at least one digit

Powtórz hasło:
••••••••

Akceptuję regulamin

Nie jestem robotem
reCAPTCHA
Prywatność - Warunki

Potwierdź, że nie jesteś botem!

Zarejestruj się



Nickname:
czarek

Istnieje już konto o takim loginie!

Twoje hasło:
••••••••

Password should contain at least one special character

Powtórz hasło:
••••••••

Akceptuję regulamin

Nie jestem robotem
reCAPTCHA
Prywatność - Warunki

Potwierdź, że nie jesteś botem!

Zarejestruj się